

Multi-Modal Human Detection System

Technical Documentation & Code Reference

HCI Assignment 4 | CLO-3

Course: Human Computer Interaction (SE305T / MD445T) — Spring-26

Institute: Information Technology University (ITU), Lahore

Instructor: Dr. Muhammad Asif

Groups: BSSE23 & BSCE22

Table of Contents

Table of Contents.....	2
1. System Overview	4
2. Architecture & Design Decisions	5
2.1 Why the New MediaPipe Tasks API?	5
2.2 Running Mode Selection.....	5
2.3 Shared Detection Loop	5
2.4 State Management via Closures	5
3. File: models.py	6
3.1 Constants	6
3.2 _download_model(url, dest).....	6
3.3 ensure_models().....	6
3.4 bgr_to_mp_image(frame)	6
4. File: utils.py	8
4.1 euclidean(p1, p2).....	8
4.2 lm_to_px(landmark, w, h)	8
4.3 compute_aspect_ratio(top, bottom, left, right).....	8
4.4 draw_text(frame, text, pos, color, scale, thickness).....	8
4.5 draw_red_border(frame, thickness=10)	9
4.6 draw_red_banner(frame, message).....	9
4.7 _esc_hint(frame).....	9
4.8 open_source(source).....	9
4.9 read_frame(cap_or_image)	9
4.10 detection_loop(source, process_fn, window_title).....	9
5. File: module_lips.py — Lips Detection.....	11
5.1 Landmark Indices	11
5.2 Thresholds.....	11
5.3 compute_mar(landmarks, w, h)	11
5.4 detect_smile(landmarks, w, h)	11
5.5 run_lips_detection(source)	11
6. File: module_eyes.py — Eyes Detection	13
6.1 Reference: Soukupova & Cech (2016)	13
6.2 Landmark Indices	13
6.3 Thresholds.....	13
6.4 compute_ear(landmarks, eye_indices, w, h).....	13
6.5 run_eyes_detection(source)	13

7. File: module_face.py — Face Detection	15
7.1 Why Three Different Detectors?	15
7.2 Landmark Indices	15
7.3 classify_head_pose(landmarks, w, h)	15
7.4 run_face_detection(source)	15
8. File: module_hand.py — Hand Detection	17
8.1 Landmark Indices	17
8.2 count_fingers(landmarks, handedness, w, h)	17
8.3 classify_gesture(fingers_up, thumb_up)	17
8.4 Gesture Map	17
8.5 run_hand_detection(source)	18
9. File: main.py — Application Entry Point	19
9.1 choose_source()	19
9.2 MODULES Registry	19
9.3 main()	19
10. Installation & Usage	20
10.1 Prerequisites	20
10.2 Running the Application	20
10.3 Navigation Quick Reference	20
11. Task Completion Summary	21

1. System Overview

The Multi-Modal Human Detection System is a real-time computer-vision application built with OpenCV and the MediaPipe Tasks API (version 0.10.x+). It processes live webcam feeds, video files, and static images to demonstrate four distinct detection modules: lip analysis, eye tracking, face recognition, and hand gesture detection.

The codebase is split into six Python files to separate concerns, improve testability, and allow modules to be imported independently in future projects.

File	Purpose
main.py	Application entry point; main menu; input-source selection
models.py	Model download management; BGR-to-MediaPipe image conversion
utils.py	Shared geometry helpers, drawing helpers, detection loop
module_lips.py	Module 1 — Lips Detection (smile, MAR, lip-sync counter)
module_eyes.py	Module 2 — Eyes Detection (blink count, EAR, drowsiness)
module_face.py	Module 3 — Face Detection (emotion, head pose, history)
module_hand.py	Module 4 — Hand Detection (finger count, gesture, game)

2. Architecture & Design Decisions

2.1 Why the New MediaPipe Tasks API?

MediaPipe 0.10.x introduced a new Tasks API that packages model weights, pre- and post-processing logic, and metadata into a single self-contained `.task` bundle. Compared to the legacy `mp.solutions.*` approach this provides:

- Explicit running modes (IMAGE vs VIDEO vs LIVE_STREAM) that enforce correct usage
- A single FaceLandmarker replaces both FaceMesh and FaceDetection
- A single HandLandmarker replaces the old Hands solution
- `drawing_utils` and `drawing_styles` inside the tasks namespace — same API, new import path

2.2 Running Mode Selection

The `RunningMode` enum controls how the landmarker handles temporal information. `IMAGE` mode applies full detection on every independent frame (no temporal filtering). `VIDEO` mode applies a Kalman-filter-like tracker across frames using a monotonically increasing timestamp, producing smoother landmarks with fewer jitter artefacts. Each module selects the correct mode at construction time by checking `isinstance(source, np.ndarray)`.

2.3 Shared Detection Loop

All four modules share the same `detection_loop()` function in `utils.py`. This loop handles: reading frames from any source type; displaying annotated frames; replaying video at end-of-file; and keyboard event handling (ESC = menu, q = quit). Each module provides a `process()` closure that encapsulates its own state and returns an annotated frame. This design eliminates code duplication and guarantees consistent keyboard behaviour across all modules.

2.4 State Management via Closures

Each `run_*` function defines a `process()` closure that captures module-specific state variables (`blink_count`, `sync_count`, `hold_start`, etc.) via the `nonlocal` keyword. This avoids global variables and class-based boilerplate while keeping all state for one module neatly scoped inside its entry function.

3. File: models.py

Handles all infrastructure concerns that every module needs but none should own: downloading the MediaPipe model bundles and converting OpenCV frames to the mediapipe.Image format.

3.1 Constants

Constant	Type	Value / Description
_MODEL_DIR	Path	~/models/ — local cache directory, auto-created
_FACE_MODEL_URL	str	Public CDN URL for face_landmarker.task (float16, v1)
_HAND_MODEL_URL	str	Public CDN URL for hand_landmarker.task (float16, v1)
FACE_MODEL_PATH	Path	_MODEL_DIR / face_landmarker.task — exported for modules
HAND_MODEL_PATH	Path	_MODEL_DIR / hand_landmarker.task — exported for modules

3.2 _download_model(url, dest)

Downloads a single .task bundle from the specified URL to the local dest path. If the file already exists the function returns immediately (no-op), making repeated application starts instant. On failure it raises RuntimeError with a manual-placement hint so users can resolve the issue without a debugger.

Parameter	Type	Description
url	str	HTTPS URL of the MediaPipe model bundle
dest	Path	Absolute local filesystem path to save the file

3.3 ensure_models()

Calls _download_model() for both the face and hand models. Designed to be called once at application startup in main() so the user sees a single, clear download progress message rather than unexpected pauses when a module first runs.

Parameters: None. Returns: None. Raises: RuntimeError propagated from _download_model().

3.4 bgr_to_mp_image(frame)

Converts an OpenCV BGR numpy array to a mediapipe.Image object with SRGB image format. This conversion is required because:

- OpenCV stores pixels in B-G-R channel order

- MediaPipe expects R-G-B channel order
- The Tasks API requires mediapipe.Image, not a raw ndarray

The conversion is: BGR ndarray → cv2.cvtColor(BGR→RGB) → RGB ndarray → MplImage(SRGB). Every module calls this function once per frame before passing the image to the landmarker.

Parameter	Type	Description
frame	np.ndarray	uint8 BGR image, shape (H, W, 3), from cv2.read() or cv2.imread()

Returns: mediapipe.Image — ready for .detect() or .detect_for_video() calls.

4. File: utils.py

Contains all geometry helpers, OpenCV drawing utilities, video-source management, and the master detection loop. No MediaPipe-specific code lives here — utils.py has no dependency on any model or task.

4.1 euclidean(p1, p2)

Computes the straight-line Euclidean distance between two 2-D points using numpy's L2 norm: $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$. Used everywhere a physical distance between landmarks is needed.

Parameter	Type	Description
p1	tuple (x,y)	First point
p2	tuple (x,y)	Second point

Returns: float — non-negative distance.

4.2 lm_to_px(landmark, w, h)

Converts a normalised MediaPipe landmark (.x, .y in [0,1]) to integer pixel coordinates by multiplying by frame width and height. Compatible with both old proto and new Tasks API landmark objects because both expose .x and .y float attributes.

Parameter	Type	Description
landmark	NormalizedLandmark	Has .x and .y in [0,1]
w	int	Frame width in pixels
h	int	Frame height in pixels

Returns: tuple (int, int) — (x_px, y_px).

4.3 compute_aspect_ratio(top, bottom, left, right)

Computes a generic vertical-to-horizontal aspect ratio: $\frac{||\text{top-bottom}||}{(||\text{left-right}|| + 1e-6)}$. A small epsilon prevents division by zero at extreme face angles. This single formula is used for both EAR (eye openness) and MAR (mouth openness) — a larger ratio means the feature is more open.

4.4 draw_text(frame, text, pos, color, scale, thickness)

Renders anti-aliased text with a one-pixel black drop-shadow. The shadow is drawn at (pos[0]+1, pos[1]+1) in black with thickness+1 stroke, then the foreground text is drawn on top at pos. This makes labels readable against any background colour or texture.

4.5 draw_red_border(frame, thickness=10)

Draws a solid red (BGR 0,0,255) rectangle around the entire frame border. Used as a warning indicator when a threshold is exceeded, for example when MAR exceeds the yawn threshold in Module 1.

4.6 draw_red_banner(frame, message)

Overlays a semi-transparent red banner (0.6 opacity) with centred white bold text at the vertical midpoint of the frame. The weighted blend preserves partial visibility of the background so the user can still see landmarks and the video feed.

4.7 _esc_hint(frame)

Draws a small greyed-out 'ESC = Menu' label in the bottom-right corner. Provides a passive on-screen reminder of the navigation shortcut without cluttering the main annotation area.

4.8 open_source(source)

Opens a cv2.VideoCapture from an integer webcam index or a file path string. Returns an already-open VideoCapture unchanged, making the function idempotent. Raises RuntimeError if the source cannot be opened.

4.9 read_frame(cap_or_image)

Reads one BGR frame from a VideoCapture via .read(), or returns a copy of a static numpy image array. Wrapping both cases in one function lets detection_loop() handle all source types with identical logic and no branching at the call site.

4.10 detection_loop(source, process_fn, window_title)

The master frame-processing loop shared by all four modules. It opens the source, creates an OpenCV window, and continuously calls process_fn(frame) then imshow(). Key bindings:

- ESC (code 27) — break the loop and return to the calling run_* function
- 'q' — release the capture, destroy all windows, raise SystemExit

For video files the stream seeks back to frame 0 at end-of-file (automatic replay). For static images cv2.waitKey(0) is used so the window stays open until any key is pressed.

Parameter	Type	Description
source	int str ndarray VideoCapture	Input source — webcam index, file path, image array, or open capture

Parameter	Type	Description
process_fn	callable	Signature: annotated_frame = process_fn(bgr_frame); must return BGR ndarray
window_title	str	OpenCV window title shown in the OS title bar

5. File: module_lips.py — Lips Detection

Implements three tasks using FaceLandmarker with the 468-point FaceMesh topology.

5.1 Landmark Indices

Constant	Index	Anatomical Location
_MOUTH_TOP	13	Upper lip center — vertical span top anchor
_MOUTH_BOTTOM	14	Lower lip center — vertical span bottom anchor
_MOUTH_LEFT	61	Left mouth corner — horizontal span anchor
_MOUTH_RIGHT	291	Right mouth corner — horizontal span anchor
_SMILE_UPPER_MID	13	Reused: upper lip Y reference for smile classification

5.2 Thresholds

Constant	Value	Meaning
MAR_OPEN_THRESH	0.55	MAR above this → mouth considered open (enter open state)
MAR_CLOSE_THRESH	0.35	MAR below this → mouth considered closed (enter closed state)

The gap between 0.55 and 0.35 is deliberate hysteresis. Without it, a MAR hovering near a single threshold would cause rapid open/close toggling and inflate the lip-sync counter.

5.3 compute_mar(landmarks, w, h)

Computes the Mouth Aspect Ratio by calling compute_aspect_ratio() with the four mouth-corner and lip-centre landmarks converted to pixel coordinates. MAR near 0 means lips pressed together; MAR above 0.55 indicates the mouth is open (yawning or loud speech).

5.4 detect_smile(landmarks, w, h)

Classifies the expression as 'Smiling' or 'Neutral'. Both mouth corners must sit below (larger Y in image coordinates) the upper lip centre by at least 5% of the inter-corner distance. This scale-normalised threshold is resolution-independent and robust to small head tilts. Empirically validated on frontal selfie photos at 0.4m–1.2m.

5.5 run_lips_detection(source)

Entry point for Module 1. Creates a FaceLandmarker, maintains hysteresis state (mouth_open, sync_count) and a synthetic timestamp, then launches detection_loop() with a process() closure that:

- T1: Calls detect_smile() and overlays the result with green (smiling) or grey (neutral)
- T2: Calls compute_mar(), displays the value; draws red border when MAR > 0.55
- T3: Updates the mouth_open boolean using hysteresis; increments sync_count on each open→close cycle

6. File: module_eyes.py — Eyes Detection

Implements blink counting, per-eye EAR display, and drowsiness detection based on the Soukupova & Cech (2016) Eye Aspect Ratio formula.

6.1 Reference: Soukupova & Cech (2016)

The EAR formula uses six landmarks per eye arranged as two vertical pairs (p2-p6 and p3-p5) and one horizontal pair (p1-p4):

$$EAR = (||p2-p6|| + ||p3-p5||) / (2 * ||p1-p4||)$$

Two vertical distances are averaged to account for the non-symmetric lid motion. Dividing by twice the horizontal span makes EAR scale-invariant — the same threshold works for faces at any distance.

6.2 Landmark Indices

Constant	Indices (p1..p6)	Eye
_LEFT_EYE_IDX	362, 385, 387, 263, 373, 380	Left eye (viewer's left)
_RIGHT_EYE_IDX	33, 160, 158, 133, 153, 144	Right eye (viewer's right)

6.3 Thresholds

Constant	Value	Meaning
EAR_THRESH	0.25	Average EAR below this = eye considered closed
DROWSY_FRAMES_ENTER	20	Consecutive closed frames required to trigger drowsy alert
DROWSY_FRAMES_CLEAR	5	Consecutive open frames required to clear the drowsy alert

6.4 compute_ear(landmarks, eye_indices, w, h)

Extracts the six landmark pixels for the specified eye, then applies the EAR formula. Returns a float; lower value = more closed. Typical open-eye range: 0.28–0.35. Typical closed range: 0.10–0.20.

6.5 run_eyes_detection(source)

Entry point for Module 2. Maintains blink_count, eye_closed flag, drowsy_frames, normal_frames, and drowsy_alert. Per-frame:

- T2: Computes left and right EAR; displays both values

- T1: If average EAR < EAR_THRESH, marks eye_closed=True; on next frame above threshold, increments blink_count
- T3: Increments drowsy_frames or normal_frames; sets/clears drowsy_alert; draws red banner when alert is active

7. File: module_face.py — Face Detection

Combines OpenCV Haar cascades, MediaPipe FaceLandmarker geometry, and DeepFace deep-learning inference to deliver emotion recognition, head pose estimation, and a rolling emotion history.

7.1 Why Three Different Detectors?

Each detector has a different strength. The Haar cascade is fast and provides a tight bounding-box ROI that DeepFace needs for accurate emotion classification. The FaceLandmarker provides 468 precise landmarks which are used for geometry-based head pose estimation. DeepFace provides semantic emotion labels that landmark geometry alone cannot deliver.

7.2 Landmark Indices

Constant	Index	Use
_NOSE_TIP	1	Primary head-pose marker; displacement from eye midpoint
_EYE_L_OUTER	33	Left eye outer corner — one anchor of the inter-eye baseline
_EYE_R_OUTER	263	Right eye outer corner — other anchor of the inter-eye baseline

7.3 classify_head_pose(landmarks, w, h)

Estimates head yaw and pitch by measuring the offset of the nose tip from the inter-eye midpoint:

- $dx = \text{nose_x} - \text{midpoint_x}$ (positive = nose moved right = head turned right)
- $dy = \text{nose_y} - \text{midpoint_y}$ (positive = nose moved down = head pitched down)
- Both dx and dy are normalised by the inter-eye distance for scale invariance
- Thresholds: $|dx| > 0.18 * \text{eye_dist} \rightarrow \text{Left/Right}$; $|dy| > 0.12 * \text{eye_dist} \rightarrow \text{Up/Down}$

The dy threshold is smaller (0.12 vs 0.18) because vertical head movement produces a smaller nose displacement than lateral rotation for the same perceived angle change.

7.4 run_face_detection(source)

Lazy-imports DeepFace so the app starts even when the library is absent. Maintains `emotion_history` as a `collections.deque` of (timestamp, emotion) tuples. Per-frame:

- T1: Every `EMOTION_INTERVAL` (5) frames, runs `DeepFace.analyze()` on the first Haar ROI; caches dominant emotion and confidence for display on all other frames
- T3: Prunes history entries older than 5 seconds; displays the most-common emotion using `collections.Counter`
- T2: Runs `FaceLandmarker`; displays classified head pose

8. File: module_hand.py — Hand Detection

Uses HandLandmarker to count fingers, classify named gestures, and drive a real-time gesture game with a hold-timer and progress bar.

8.1 Landmark Indices

Constant	Values	Purpose
<code>_FINGER_TIPS</code>	4, 8, 12, 16, 20	Fingertip landmarks: thumb, index, middle, ring, pinky
<code>_FINGER_PIPS</code>	3, 6, 10, 14, 18	PIP joint landmarks used as the 'bent' reference point

8.2 `count_fingers(landmarks, handedness, w, h)`

Determines which fingers are extended. Non-thumb fingers: tip Y < PIP Y means extended (fingertip is above the knuckle in the image). Thumb: uses X-axis because it extends sideways. Chirality: right hand tip_x < PIP_x; left hand tip_x > PIP_x. Returns (non_thumb_count: int, thumb_up: bool).

Tip_x < num = left

Tip_x > num = right

Tip_y < num = up

Tip_y > num = down

8.3 `classify_gesture(fingers_up, thumb_up)`

Looks up (fingers_up, thumb_up) in `_GESTURE_MAP`. Falls back to (fingers_up, False) for configurations where the thumb state is ambiguous. Returns (gesture_label, short_tag). The short tag (e.g. 'V' for Peace, '+1' for Thumbs Up) is displayed in brackets next to the gesture name.

8.4 Gesture Map

fingers_up	thumb_up	Gesture	Tag
0	False	Fist	OK
0	True	Thumbs Up	+1
1	False	One	1
2	False	Peace	V

fingers_up	thumb_up	Gesture	Tag
3	False	Three	3
4	False	Four	4
4	True	Open Hand	HI
5	True	High Five	5

8.5 run_hand_detection(source)

Entry point for Module 4. Detects up to 2 hands simultaneously. For each hand: draws the skeleton, calls count_fingers(), calls classify_gesture(), annotates near the wrist. Game logic (T3): picks a random target from _GAME_GESTURES; when the target appears in current_gestures, starts hold_start timer; draws a progress bar (width = hold_fraction * 200 px); on completion increments score and picks a new target.

9. File: main.py — Application Entry Point

9.1 choose_source()

Console sub-menu that returns the correct Python type for each input source: integer 0 for webcam, string path for video, np.ndarray for static image. Returning different types is intentional — `detection_loop()` uses `isinstance()` to select the correct frame-reading and timing strategy.

Choice	Returns	Detection Loop Behaviour
1 (Webcam)	int (0)	cv2.VideoCapture(0); waitKey(30); replays on EOF
2 (Video)	str	cv2.VideoCapture(path); waitKey(30); replays on EOF
3 (Image)	np.ndarray	Single frame; waitKey(0); exits after keypress
0 (Back)	None	Skips module dispatch; loops to main menu

9.2 MODULES Registry

A plain dict mapping key strings '1'–'4' to (display_name, run_function) tuples. Adding a new module requires only adding one entry here and creating the corresponding `run_*` function — the menu loop and dispatch code require no changes.

9.3 main()

Prints the application banner, calls `ensure_models()` to cache model files, then enters a loop: print menu → read choice → validate → call `choose_source()` → dispatch to module. `RuntimeError` (source open failure) and `KeyboardInterrupt` are caught and printed without crashing the loop, allowing the user to try again.

10. Installation & Usage

10.1 Prerequisites

Package	Install Command	Notes
Python 3.9+	—	Required
OpenCV	pip install opencv-python	Webcam + drawing
MediaPipe 0.10.x+	pip install mediapipe	Tasks API
NumPy	pip install numpy	Array ops
DeepFace	pip install deepface	Optional — Module 3 emotion
TensorFlow / tf	pip install tensorflow	Required by DeepFace

10.2 Running the Application

```
python main.py
```

On first run, models are downloaded automatically to the current_folder/model/. Subsequent starts are instant.

10.3 Navigation Quick Reference

Context	Key	Action
Main menu	1–4	Select detection module
Main menu	q	Quit application
Source menu	1	Open default webcam
Source menu	2	Enter video file path
Source menu	3	Enter image file path
Source menu	0	Back to main menu
Any window	ESC	Return to main menu
Any window	q	Quit application immediately

11. Task Completion Summary

Module	Task	Implementation	Key Threshold / Logic
Lips	T1 Smile detection	detect_smile()	Corner drop > 5% of mouth width
Lips	T2 MAR tracking	compute_mar()	MAR > 0.55 → red border
Lips	T3 Lip-sync counter	Hysteresis state machine	Open>0.55, Close<0.35
Eyes	T1 Blink counter	EAR threshold crossing	avg EAR < 0.25 then rises
Eyes	T2 EAR display	compute_ear()	Soukupova & Cech formula
Eyes	T3 Drowsiness alert	Frame counters	20 closed frames → alert
Face	T1 Emotion recognition	DeepFace.analyze()	Every 5 frames on Haar ROI
Face	T2 Head pose	classify_head_pose()	Nose offset vs eye midpoint
Face	T3 Emotion history	collections.deque (5s)	Counter.most_common(1)
Hand	T1 Finger counter	count_fingers()	Tip Y vs PIP Y + chirality
Hand	T2 Gesture label	classify_gesture()	_GESTURE_MAP lookup
Hand	T3 Gesture game	hold_start timer	1-second hold → 1 point